

Esercizio 1 – Calcola

Si scriva una classe `Elemento` che rappresenta un nodo di una lista concatenata. Il costruttore di `Elemento` prende in input un valore (numero), da assegnare al campo `val`, e setta il successore dell'elemento (memorizzato nel campo `next`) a `undefined`.

Scrivere poi un generatore `calcola(testa,f)` che prende in input la testa di una lista e una funzione `f`. Ad ogni elemento incontrato, il generatore restituisce il prodotto di tutti i valori ottenuti applicando `f` al valore contenuto nell'elemento corrente e nei suoi predecessori.

In simboli, se la lista contiene i valori $v_1, v_2, \dots, v_k$, il generatore produce:

- $f(v_1)$
- $f(v_1) + f(v_2)$
- $f(v_1) + f(v_2) + f(v_3)$
- ...
- $f(v_1) + f(v_2) + \dots + f(v_k)$

```
class Elemento {
  constructor(val) {
    this.val = val;
    this.next = undefined;
  }
}

function* calcola(testa, f) {
  var cur = testa;
  var acc = 0;

  while (cur !== undefined) {
    acc = acc + f(cur.val);
    yield acc;
    cur = cur.next;
  }
}
```

Esercizio 2 – Consegne pacchi

Si implementi un sistema per il tracciamento dello stato di spedizioni.

Si scriva una classe `Pacco` con i seguenti campi:

- un identificatore (stringa),
- un peso (numero),
- uno stato (stringa) tra "CREATO", "IN_TRANSITO", "CONSEGNATO", e "BLOCCATO", e
- una sequenza cronologica di eventi di cambiamento di stato, ciascuno modellato come una coppia `[dataOraCambiamento, statoRaggiunto]`, con `dataOraCambiamento` istanza della classe `Date`.

Tutti i campi devono essere accessibili in sola lettura, ad eccezione del campo `peso` che è accessibile in lettura e scrittura. Le modifiche di `peso` sono però consentite solo nello stato "CREATO"; se un pacco non è in stato "CREATO", un tentativo di modifica deve causare il lancio dell'eccezione `ModificaNonConsentita`, definita in modo da tenere traccia (come campi aggiuntivi) di `id` e `stato` del pacco che l'ha causata.

La classe `Pacco` deve fornire i seguenti metodi:

- un costruttore che consente di specificare identificatore e peso della nuova spedizione creata;
- un metodo `avanza(stato)` che aggiorna lo stato del pacco e registra il relativo cambiamento di stato.

Se il valore di `stato` passato al metodo `avanza` non è uno degli stati ammessi, il metodo deve lanciare un'eccezione custom `StatoNonValido`.

Si scriva una classe `PaccoRefrigerato` che estende `Pacco` e introduce due campi `temperatura_minima` e `temperatura_massima`, accessibili in sola lettura, e un campo `temperatura_attuale`. Il costruttore di `PaccoRefrigerato` verifica che `temperatura_minima` sia inferiore a `temperatura_massima`, altrimenti lancia un `RangeError`. Quando viene creata e modificata la `temperatura_attuale`, la classe `PaccoRefrigerato` verifica che si trovi nell'intervallo `[temperatura_minima, temperatura_massima]`; se questo non si verifica, lo stato viene aggiornato in "BLOCCATO" e viene lanciata un'eccezione `CatenaDelFreddoRotta`, definita opportunamente.

Si scriva una classe `PaccoFreezer`, per rappresentare un pacco refrigerato con temperatura massima a 0 gradi.

Si scrivano infine due funzioni:

- `aggiorna(pacchi_refrigerati, temperatura)` che, dato un array di `pacchi_refrigerati`, aggiorna la loro `temperatura_attuale` alla temperatura passata e restituisce un nuovo array contenente tutti i pacchi la cui catena del freddo è stata rotta, nello stesso ordine in cui compaiono nell'array `pacchi_refrigerati`;
- si scriva infine un generatore `bloccati(pacchi)` che, dato un array di pacchi, restituisce i pacchi non refrigerati in stato "BLOCCATO", uno dopo l'altro e nello stesso ordine in cui compaiono nell'array `pacchi`.

Commentare opportunamente il codice.

```
class ModificaNonConsentita extends Error {constructor(m) { super(m);}}
class StatoNonValido extends Error {constructor(m) { super(m);}}
class CatenaDelFreddoRotta extends Error {constructor(m) { super(m);}}

class Pacco {
  static STATI = new Set(["CREATO", "IN_TRANSITO", "CONSEGNATO", "BLOCCATO"]);
  #id;
  #peso;
  #stato;
  #eventi;

  constructor(id, peso) {
    this.#id = id;
    this.#peso = peso;
    this.#stato = "CREATO";
    this.#eventi = [];
  }

  get id() { return this.#id; }
  get stato() { return this.#stato; }
  get eventi() { return this.#eventi; }

  get peso() { return this.#peso; }
  set peso(v) {
    if (this.#stato !== "CREATO") {
      var e = new ModificaNonConsentita("peso");
      e.id = this.#id;
      e.stato = this.#stato;
      throw e;
    }
    this.#peso = v;
  }

  avanza(stato) {
    if (!Pacco.STATI.has(stato)) throw new StatoNonValido("stato");
    this.#stato = stato;
    this.#eventi.push([new Date(), stato]);
  }
}

class PaccoRefrigerato extends Pacco {
  #tmin;
  #tmax;
  #tatt;

  constructor(id, peso, temperatura_minima, temperatura_massima, temperatura_attuale) {
    super(id, peso);
    if (!(temperatura_minima < temperatura_massima)) throw new RangeError("range");

    this.#tmin = temperatura_minima;
    this.#tmax = temperatura_massima;

    // set iniziale, con controllo catena del freddo
  }
}
```

```

    this.temperatura_attuale = temperatura_attuale;
}

get temperatura_minima() { return this.#tmin; }
get temperatura_massima() { return this.#tmax; }

get temperatura_attuale() { return this.#tatt; }
set temperatura_attuale(v) {
    this.#tatt = v;

    if (v < this.#tmin || v > this.#tmax) {
        // aggiorna stato e registra evento
        super.avanza("BLOCCATO");
        throw new CatenaDelFreddoRotta("freddo");
    }
}
}

class PaccoFreezer extends PaccoRefrigerato {
    constructor(id, peso, temperatura_minima, temperatura_attuale) {
        super(id, peso, temperatura_minima, 0, temperatura_attuale);
    }
}

function aggiorna(pacchi_refrigerati, temperatura) {
    var rotti = [];
    for (var i = 0; i < pacchi_refrigerati.length; i++) {
        var p = pacchi_refrigerati[i];
        if (!(p instanceof PaccoRefrigerato)) continue;

        try {
            p.temperatura_attuale = temperatura;
        } catch (e) {
            if (e instanceof CatenaDelFreddoRotta) rotti.push(p);
        }
    }
    return rotti;
}

//function* freddiRotti(pacchi) {
function* bloccati(pacchi) {
    for (var i = 0; i < pacchi.length; i++) {
        var p = pacchi[i];
        if (!(p instanceof Pacco)) continue;
        if (!(p instanceof PaccoRefrigerato) && p.stato === "BLOCCATO") {
            yield p;
        }
    }
}
}

```

Esercizio 3 – Caveau

Si scriva una classe JavaScript Caveau che rappresenta un portafoglio con saldo protetto.

Il costruttore prende due argomenti:

- owner, una stringa non vuota;
- saldoIniziale, un numero maggiore o uguale a 0 (opzionale, valore di default 0).

Se i parametri non sono validi, il costruttore deve lanciare un'eccezione di tipo TypeError.

La classe deve consentire l'accesso in sola lettura al saldo disponibile e offrire i seguenti metodi:

- versa(n, causale) che aggiunge n al saldo;
- preleva(n, causale) che sottrae n dal saldo;
- estratto(k) che restituisce gli ultimi k movimenti effettuati (default k = 10), dal più recente al meno recente.

I metodi versa e preleva devono lanciare un `TypeError` se `n` non è un numero positivo o causale non è una stringa. Il metodo preleva deve inoltre lanciare un'eccezione `FondiInsufficienti` se `n` è maggiore del saldo disponibile.

Ogni movimento è un oggetto caratterizzato da:

- tipo ("V" = versamento o "P" = prelievo);
- importo (numero ≥ 0);
- causale (stringa).

I campi tipo, importo e causale non devono essere modificabili.

La classe `Caveau` deve inoltre offrire un metodo statico `transazioni()` che restituisce un insieme di tutte le transazioni effettuate su tutte le istanze di `Caveau` create, dove ogni transazione è rappresentata da una coppia `[c,m]`, con `c` che è un riferimento all'istanza del `Caveau` e `m` che è un movimento rappresentato come descritto sopra.

Commentare opportunamente il codice.

```
class FondiInsufficienti extends Error {
  constructor(m) {
    super(m);
  }
}

class Caveau {
  static #tutteTransazioni = [];

  #owner;
  #saldo;
  #movimenti;

  constructor(owner, saldoIniziale = 0) {
    if (typeof owner !== "string" || owner.length === 0) throw new TypeError("owner");
    if (typeof saldoIniziale !== "number" || Number.isNaN(saldoIniziale) ||
saldoIniziale < 0) {
      throw new TypeError("saldoIniziale");
    }
    this.#owner = owner;
    this.#saldo = saldoIniziale;
    this.#movimenti = [];
  }

  get saldo() {
    return this.#saldo;
  }

  versa(n, causale) {
    if (typeof n !== "number" || Number.isNaN(n) || n <= 0) throw new TypeError("n");
    if (typeof causale !== "string") throw new TypeError("causale");

    this.#saldo += n;
    this.#registra("V", n, causale);
  }

  preleva(n, causale) {
    if (typeof n !== "number" || Number.isNaN(n) || n <= 0) throw new TypeError("n");
    if (typeof causale !== "string") throw new TypeError("causale");
    if (n > this.#saldo) throw new FondiInsufficienti("fondi");

    this.#saldo -= n;
    this.#registra("P", n, causale);
  }

  estratto(k = 10) {
    var out = [];
    if (k <= 0) {return out;}
  }
}
```

```

    // Copio gli oggetti interni
    var slice = this.#movimenti.slice(-k).reverse();
    for (var i = 0; i < slice.length; i++) {
        var m = slice[i];
        out.push({ tipo: m.tipo, importo: m.importo, causale: m.causale });
    }
    return out;
}

static transazioni() {
    // Restituisce un Set di coppie [c, m], con m copiato (c rimane il riferimento al
    caveau).
    var s = new Set();
    for (var i = 0; i < Caveau.#tutteTransazioni.length; i++) {
        var pair = Caveau.#tutteTransazioni[i];
        var c = pair[0];
        var m = pair[1];
        s.add([c, { tipo: m.tipo, importo: m.importo, causale: m.causale }]);
    }
    return s;
}

#registra(tipo, importo, causale) {
    var m = { tipo: tipo, importo: importo, causale: causale };
    this.#movimenti.push(m);
    Caveau.#tutteTransazioni.push([this, m]);
}
}

```